

HCCL基础概念和典型算子、算法介绍

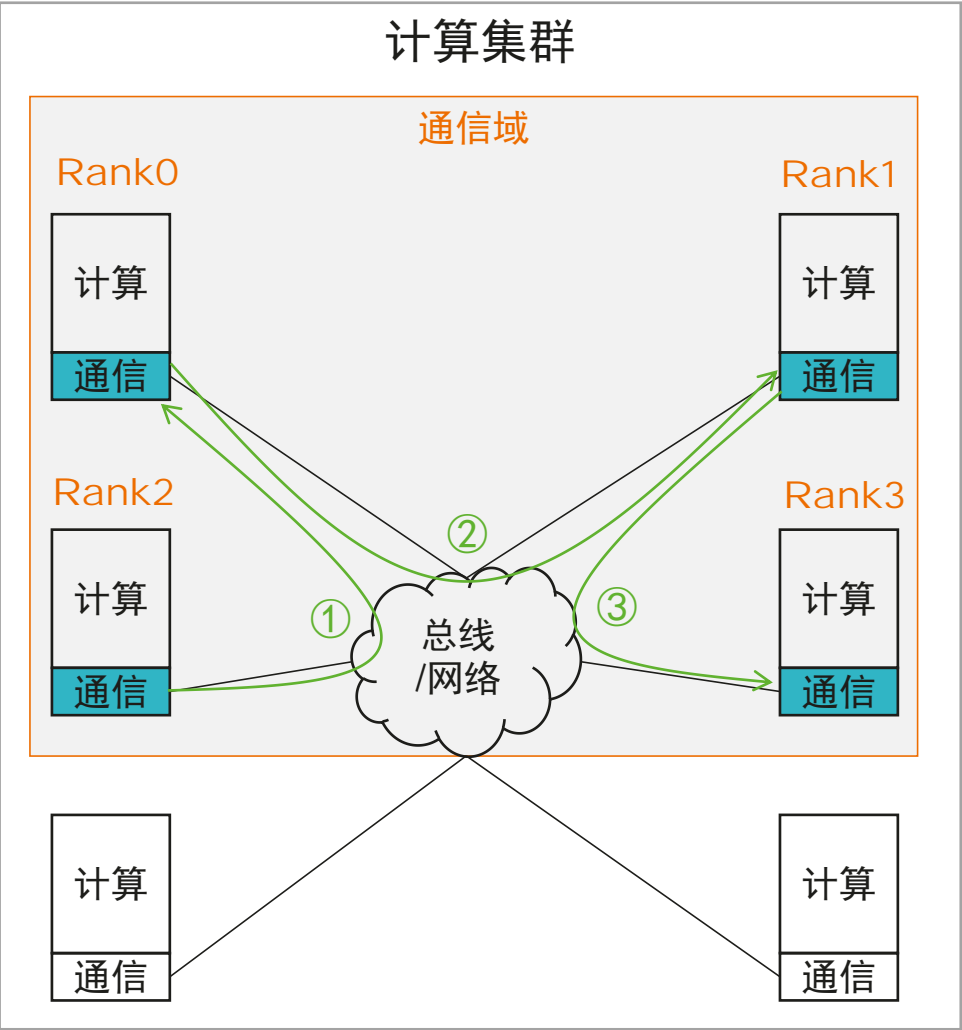
目录

Part 1 HCCL基础概念

Part 2 HCCL算子介绍

Part 3 HCCL算法简介

HCCL基础概念：通信域组织通信成员，通信算子承载通信算法



通信成员：参与通信的最小逻辑实体，可以是一颗芯片、一个Die、甚至一个进程，一般称为Rank。

通信域：一组通信成员的组合，描述通信范围。一个计算集群可创建多个通信域，一个通信成员可加入多个通信域。

通信算子：在通信域内完成通信任务的算子，集合通信指所有成员一起参与的通信操作，如Broadcast。通信域内通信算子串行执行。

通信算法：完成通信算子的实现方案，不同拓扑、数据量下可能有不同的通信算法方案，实现不同流量规划策略。一个通信算法中一般包括多个通信步骤。

通信域

通信算子

Ring通信算法

目录

Part 1 HCCL基础概念

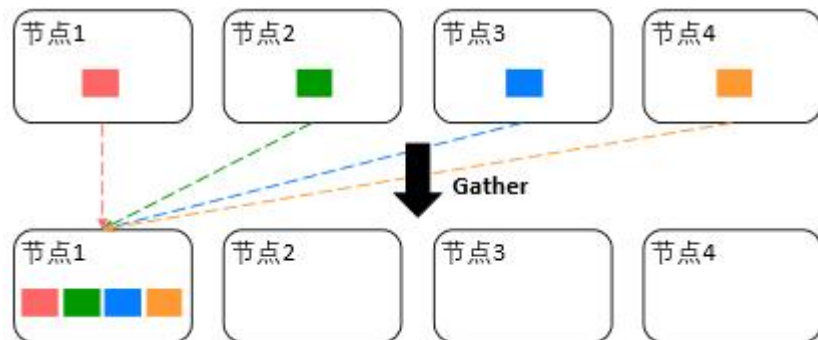
Part 2 HCCL算子介绍

Part 3 HCCL算法简介

常见集合通信算子：多对一、一对多

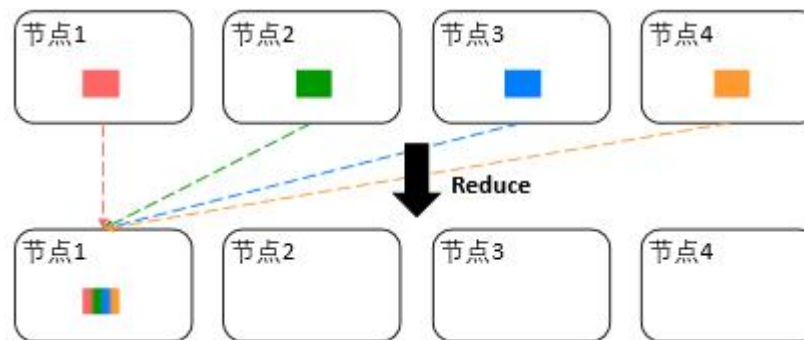
Gather:

收集通信域内所有节点的数据到root节点



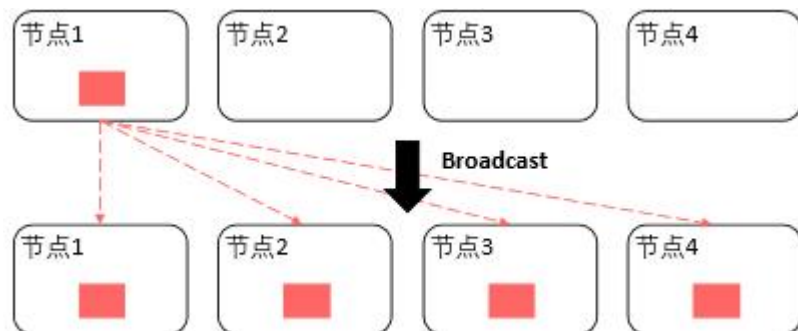
Reduce:

收集通信域内所有节点的数据到root节点并做Reduce操作



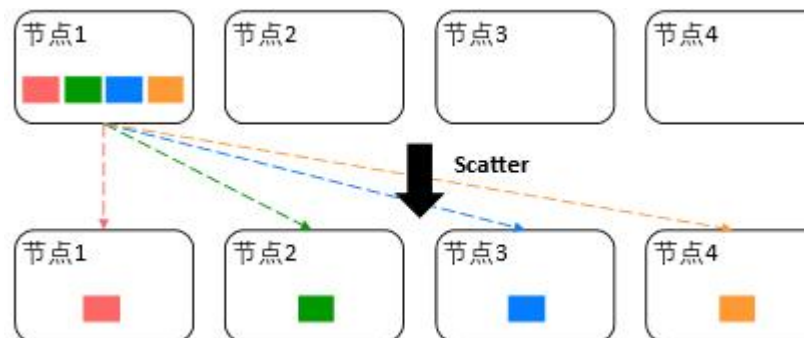
Broadcast:

将通信域内root节点的数据广播到所有节点



Scatter:

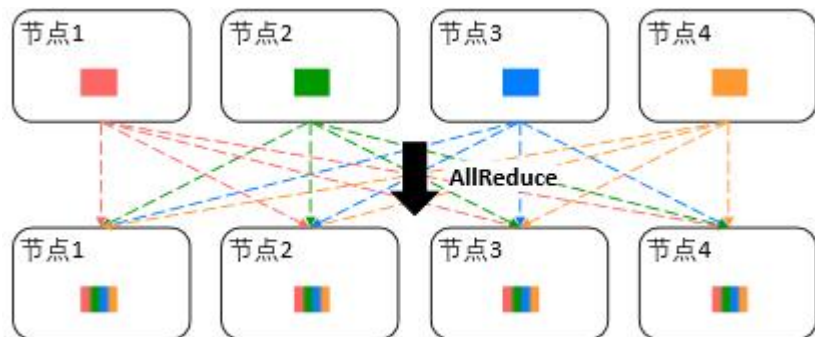
将通信域内root节点的数据均分至所有节点



常见集合通信算子：多对多

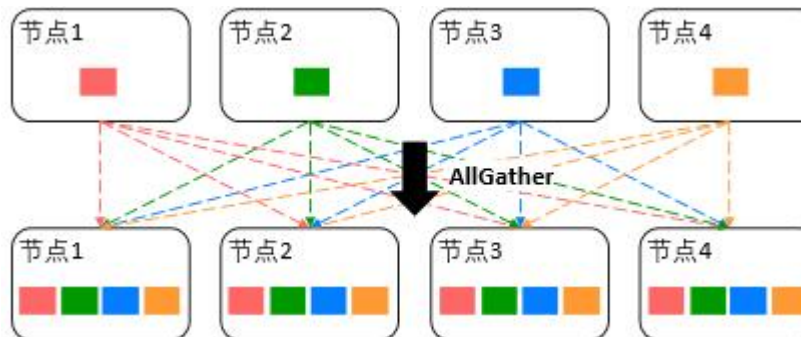
AllReduce:

每个节点收集通信域内所有节点的数据并做Reduce操作



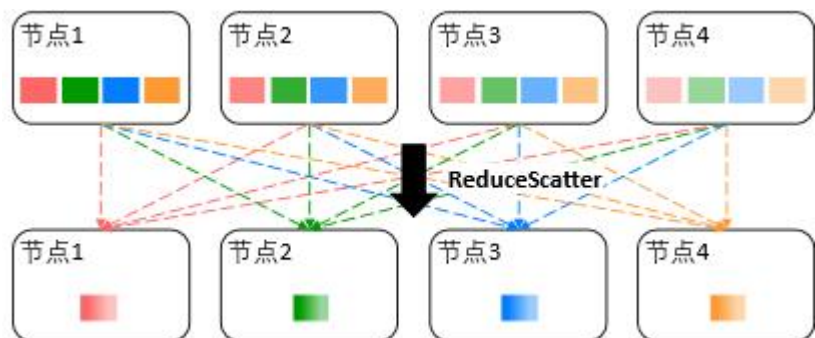
AllGather:

每个节点收集通信域内所有节点的数据



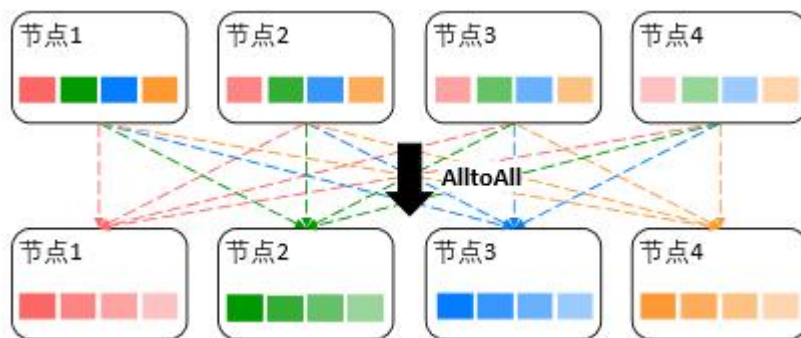
ReduceScatter:

将通信域内所有节点数据做Reduce操作后均分至所有节点

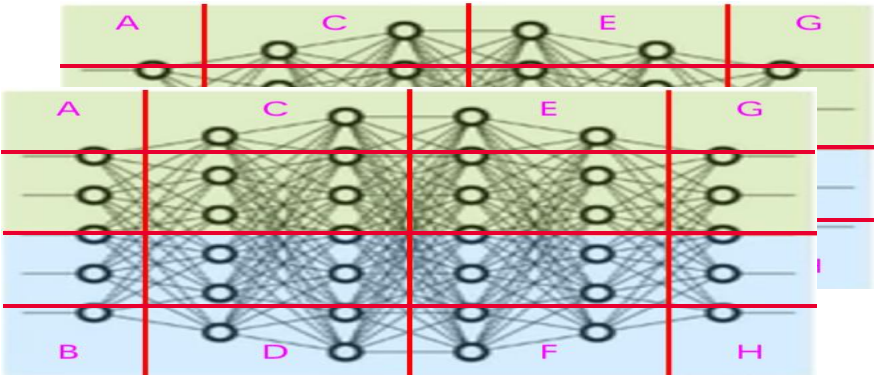


AlltoAll:

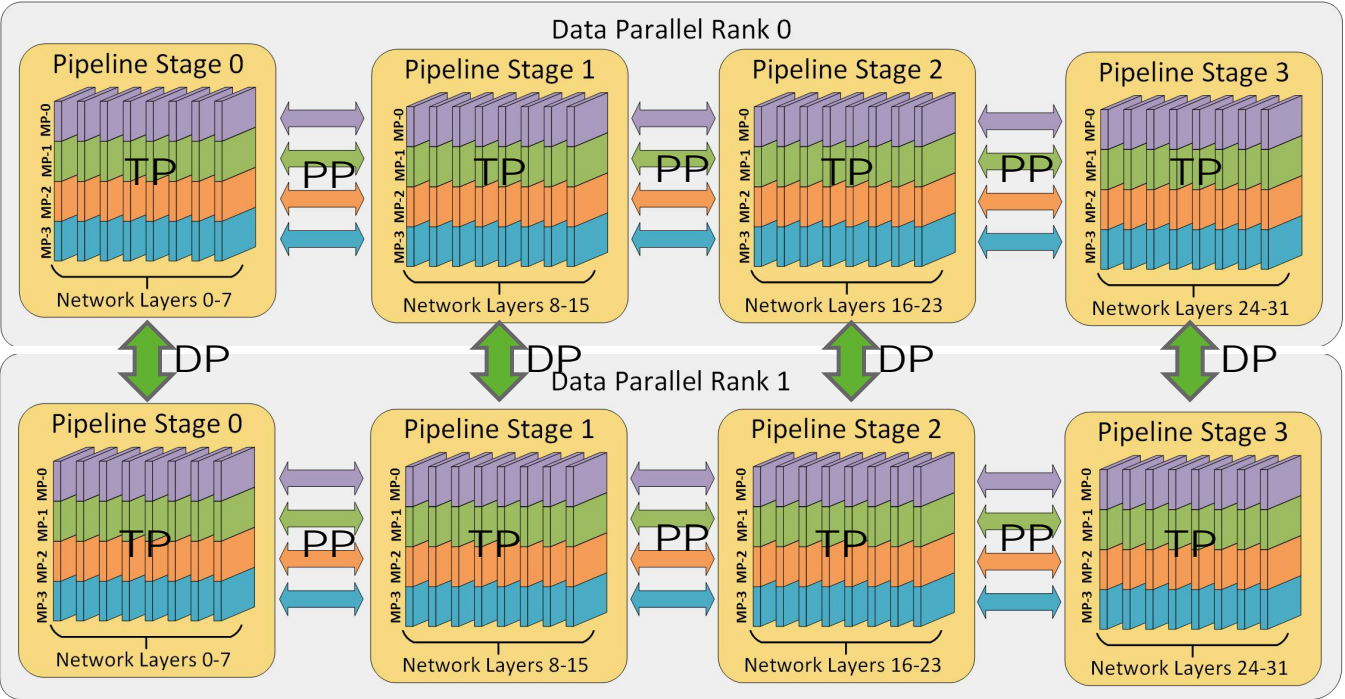
通信域内所有节点相互scatter和gather等量的数据



大模型的典型通信模型：通信可以被计算部分隐藏



模型切分



类型	操作类型	节点规模	数据量	备注
张量并行(TP)	allreduce、allgather、reducescatter	Max to 64 Chip	百MB ~GB	Matmul流水，可隐藏部分通信
流水并行(PP)	Send/Recv(点到点通信)	2 Chip	十MB ~ 百MB	可流水隐藏
数据并行(DP)	allreduce	Max to 1000+ Chip	GB	计算通信overlap，可隐藏
专家并行(EP)	all2all	Max to 256 Chip	MB ~ 百MB	计算通信串行，不可隐藏

课堂小练

1、（连线题）：常见的集合通信算子

按维度执行归约并发送到对应NPU上

收集每个NPU所有的数据到每个NPU上

所有NPU互相scatter和gather数据

从多个NPU获取数据到所有NPU并做归约运算

AllGather

AllReduce

All2All

ReduceScatter

2、（单选题）稀疏MoE模型分布式训练中，“将数据分发给对应专家、从对应专家接收计算后的结果”这一Dispatch-Combine流程，使用到的集合通信算子类别是？：

- A) AllReduce算子
- B) AllGather类算子
- C) AlltoAll类算子
- D) ReduceScatter类算子

课堂小练（解答）

1、（连线题）：常见的集合通信算子

按维度执行归约并发送到对应NPU上

收集每个NPU所有的数据到每个NPU上

所有NPU互相scatter和gather数据

从多个NPU获取数据到所有NPU并做归约运算

AllGather

AllReduce

All2All

ReduceScatter

2、（单选题）稀疏MoE模型分布式训练中，“将数据分发给对应专家、从对应专家接收计算后的结果”这一Dispatch-Combine流程，使用到的集合通信算子类别是：

- A) AllReduce算子
- B) AllGather类算子
- C) AlltoAll类算子
- D) ReduceScatter类算子

C

目录

Part 1 HCCL基础概念

Part 2 HCCL算子介绍

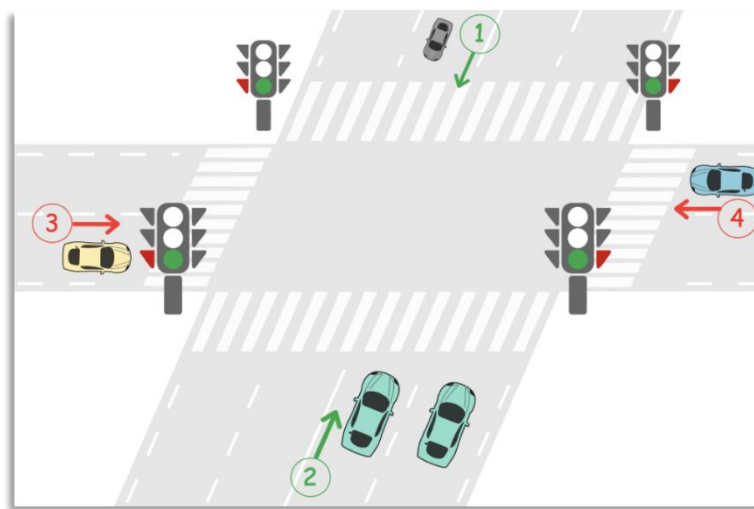
Part 3 HCCL算法简介

为什么需要集合通信算法？

没有红绿灯：各种拥塞



需要交通枢纽的管控和规划



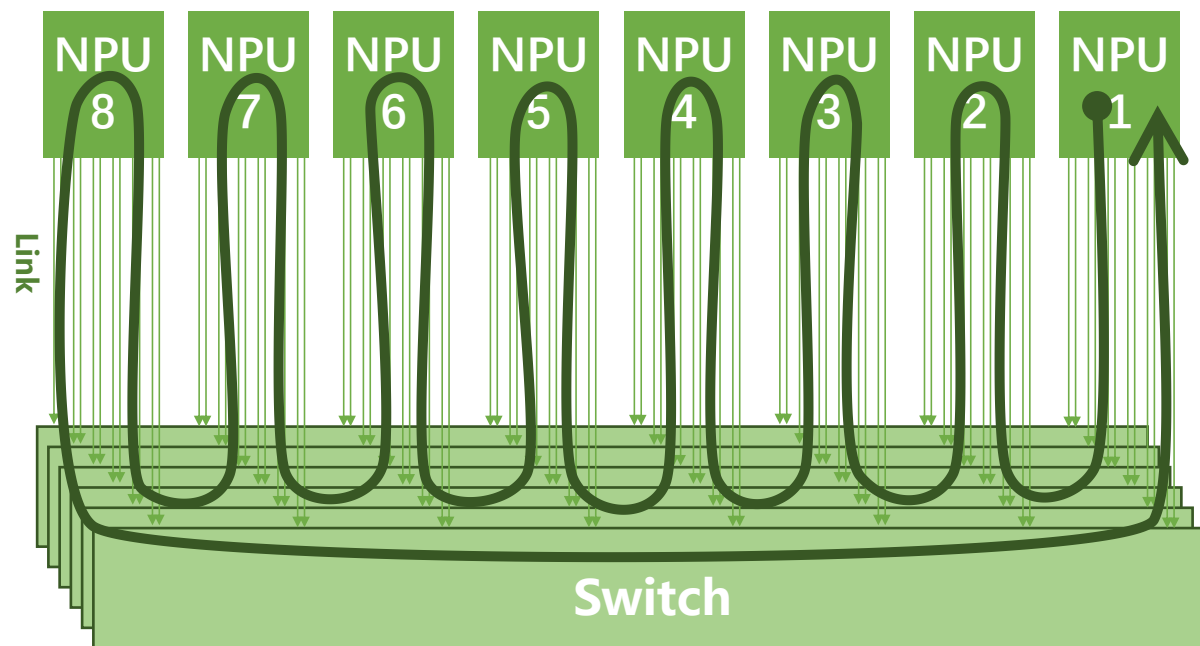
需求：集群中1万张卡，做一次求平均

思考：

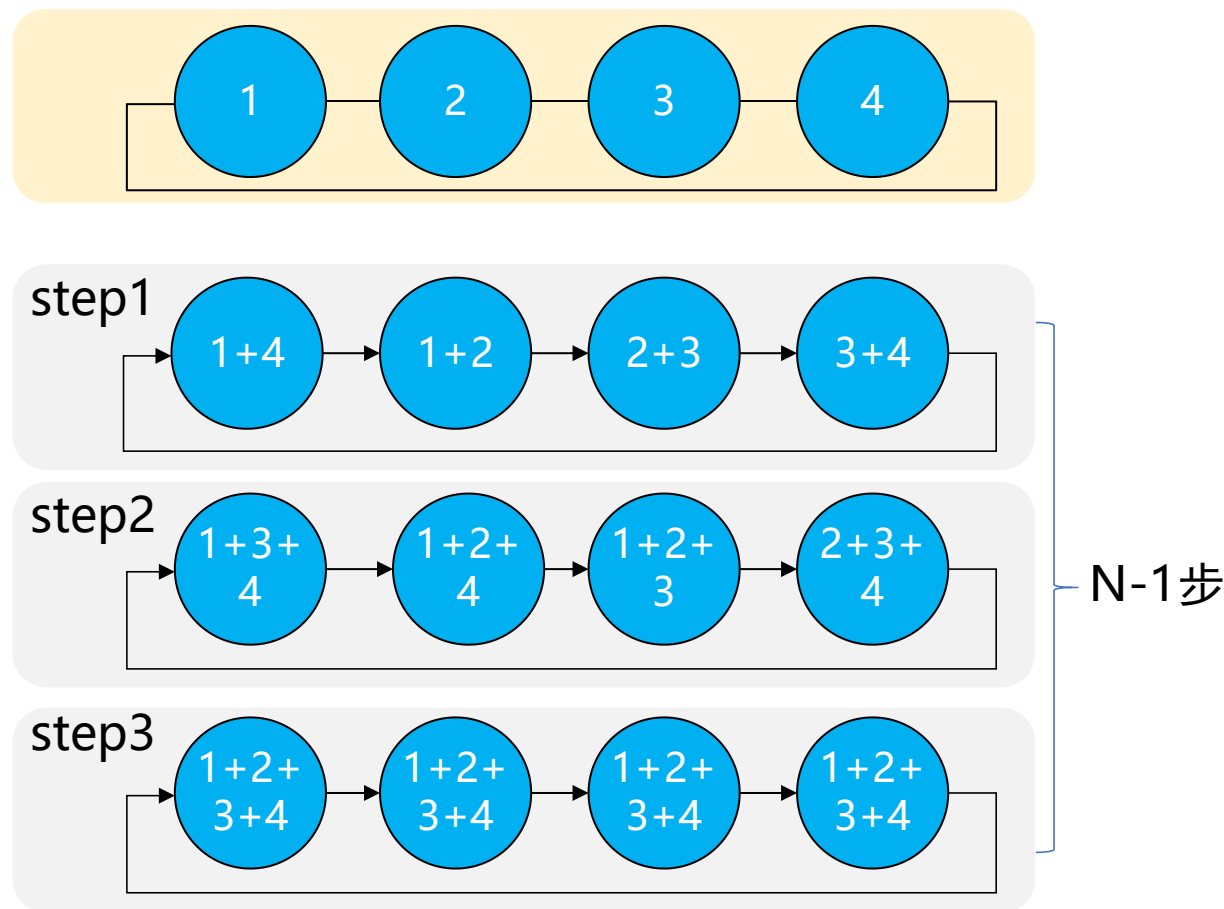
如果没有统一流量规划，流量会怎样？

集合通信库实现**流量统一规划**，满足复杂物理拓扑中**流量有序交换**，最大化集群通信性能

常用算法介绍：Ring算法

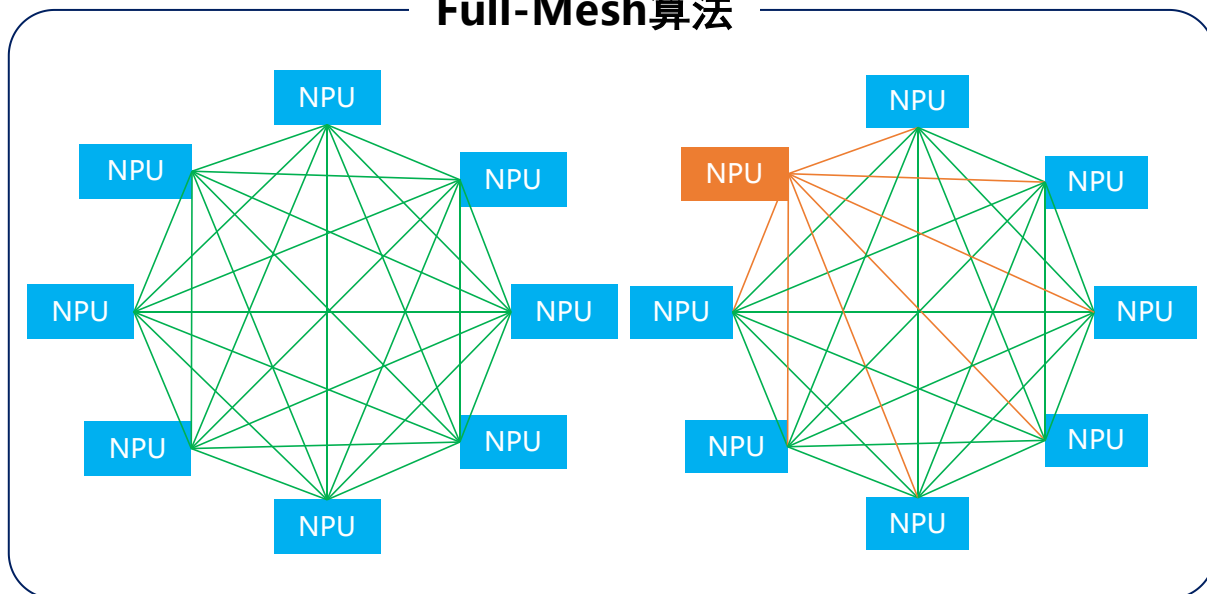


- 适用拓扑：Star、Tree等规模
- 通信步骤：N-1



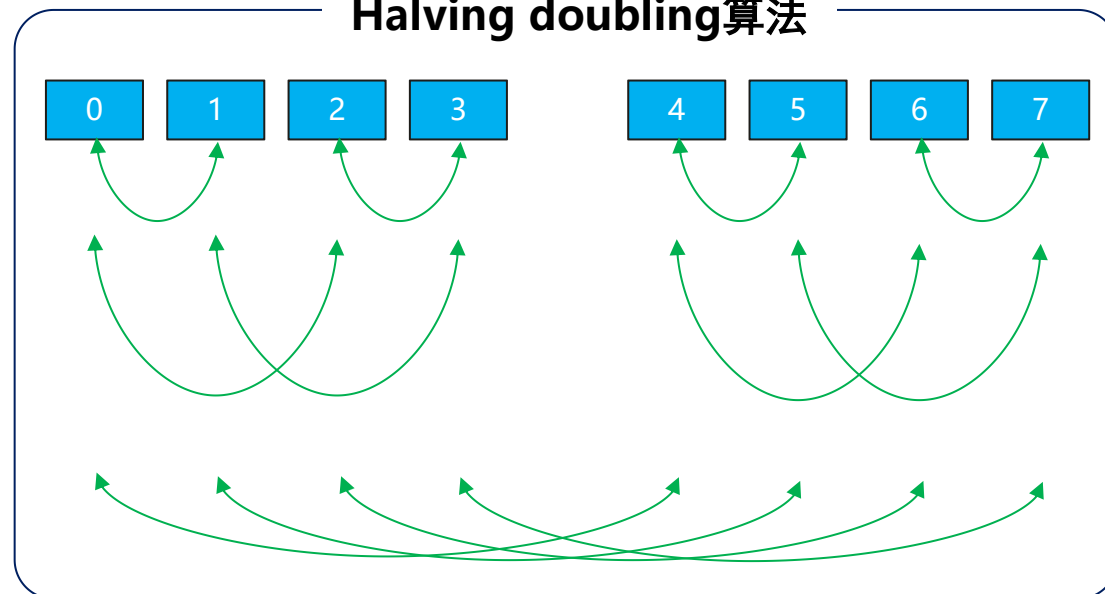
常用算法介绍：Mesh算法 & Halving-Doubling算法

Full-Mesh算法



- 适用拓扑：Full-mesh
- 通信步骤：仅需1步
- 同时并发访问其它所有NPU

Halving doubling算法



- 适用拓扑：Fat-Tree等
- 通信步骤： $\log_2 N$
- 访问步长按照 2^N 增长

总结

- **HCCL算子**
 - 一对多、多对一
 - 多对多
- **HCCL算法**
 - MESH、RING、Halving-Doubling
 - 实现复杂物理拓扑中流量有序交换，最大化集群通信性能

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.



上CANN社区获取干货



关注CANN公众号获取资讯

<https://gitcode.com/cann>

CANN